

No. My SQL Questions

What is MySQL

What is MySQL?

What is Sql?

What is the difference between SQL and MySQL?

No. Database

What is a database?

Database:- Show

Database:- Create

Database:- Rename

Database:- Drop/Delete

Database:- Select

Difference between CHAR vs VARCHAR

SQL Comments?

What is Aggregate function?

No. Keys

Primary Key?

primary Key:- Add

primary Key:- Delete

Unique Key?

ALTER unique key?

Drop unique key?

Difference between Primary Key & Unique Key?

Foreign Key?

Foreign Key Add/ALTER?

DROP Foreign Key?

Composite Key?

Difference between Primary Key & Foreign Key?

No. Joins

What Is Joins?

self join

No. Joins

INNER JOIN

Left JOIN/LEFT OUTER JOIN

Right JOIN

Outer join

CROSS Join

Full Join/FULL OUTER JOIN

What is MySQL?

- MySQL is an open-source relational **database management system** (RDBMS) that uses SQL to store, manage, and retrieve data.
- It's commonly used for managing data in web applications and is known for its performance and ease of use.

What is Sql?

- SQL stands for **structure query language**.
- SQL (Structured Query Language) is a standard language used to **create, read, update, and delete** data in relational databases. It is also used to create and modify database structures such as tables.
- SQL language hai, database nahi.

What is the difference between SQL and MySQL?

- SQL is a standard language used to communicate with relational databases, whereas MySQL is an RDBMS that uses SQL to store, manage, and retrieve data.
- SQL → Language
- MySQL → RDBMS / Database Management System

What is a database?

- A database is an organized collection of data that allows efficient storage, retrieval, and management.

Show Database

```
SHOW DATABASES/SCHEMAS;
SHOW DATABASES [LIKE 'pattern' | WHERE expr];
SHOW DATABASES LIKE 'test%';
SELECT * FROM sys.databases;
EXEC sp_databases;

+-----+
| Database          |
+-----+
| employeesdb       |
| profile_fastapi   |
+-----+
```

Create Database

```
CREATE DATABASE databasename;
```

Rename Database

```
RENAME DATABASE old_database_name TO new_database_name
(OR)
ALTER DATABASE old_database MODIFY NAME = new_database
```

Drop Database

```
DROP DATABASE/SCHEMA database_name;

-- DROP DATABASE IF EXISTS Statement
DROP DATABASE IF EXISTS DatabaseName;

-- Deleting Multiple Databases
DROP DATABASE testDB3, testDB4;
```

Select Database

```
USE YourDatabaseName;
```

Difference between CHAR vs VARCHAR

- Both of these data types are used for characters.
- CHAR is a **fixed-length data type**, whereas VARCHAR is a **variable-length data type**.
- CHAR is suitable for **fixed-size values**, while VARCHAR is suitable for **values whose length can vary**.
- char occupies all the space and if space is remaining, then it fill all the blank space with "space". But in case of varchar, It takes only the required length & release remaining.

```
Char -> 10      | R | A | M | space | space | sapce | space | space | space |
space |
Varchar -> 10   | R | A | M |   |   |   |   |   |   |
| R | A | M |
```

- varchar is better than Char in term of space.
- char perform is better than varchar.
- Char max 256 characters, varchar 65535 characters.

```
-- CHAR Example
CREATE TABLE users (
    country_code CHAR(2)
);

INSERT INTO users (country_code)
VALUES ('IN'), ('US'), ('UK');

-- Yahan CHAR(2) suitable hai kyunki har country code ki length fixed 2 characters
hai.

--
-- varchar example
-- jaise name
CREATE TABLE users (
    name VARCHAR(50)
);
```

SQL Comments?

- There are typically two main types of SQL comments:

1. **Single-line Comments:** Started with two dashes (--)

```
-- This is a single-line comment  
SELECT * FROM employees; -- Retrieve all employee records
```

2. **Multi-line Comments:** Started with /* and ended with */

```
/* This is a  
   multi-line comment  
   explaining the query */  
SELECT id, name, email FROM employees;
```

What is Aggregate function?

```
-- Sum() :- The SUM() function returns the total sum of a numeric column.  
SELECT SUM(column_name) FROM table_name;  
  
-- AVG() :- The AVG() function returns the average value of a numeric column.  
SELECT AVG(column_name) FROM table_name;  
  
-- MAX() :- The MAX() function returns the largest value of the selected column.  
SELECT MAX(column_name) FROM table_name;  
  
-- Min() :- The MIN() function returns the smallest value of the selected column.  
SELECT MIN(column_name) FROM table_name;  
  
-- count() :- The COUNT() function returns the number of rows that matches a  
specified criterion.  
SELECT COUNT(column_name) FROM table_name;
```


Primary Key?

- A PRIMARY KEY is a column or combination of columns that **uniquely identifies each record** in a database table.
- A Primary Key column **cannot have Null values**.
- A table can have only **one primary key** per table.
- When **multiple fields** are used as a primary key, they are called a **composite key**.

```
-- Create Primary Key
CREATE TABLE Students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100)
);
-- (OR)
CREATE TABLE Students (
    student_id INT AUTO_INCREMENT,
    name VARCHAR(50),
    email VARCHAR(100),
    PRIMARY KEY (student_id)
);

-- Create Primary Key with multiple column
CREATE TABLE Order_Items (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id) -- Multiple columns as primary key
);
```

Add primary Key

```
-- if primary key doesn't exists in the created table
ALTER TABLE table_name ADD PRIMARY KEY (Id)

-- For multiple column
ALTER table Employee ADD constraints PK_Employee PRIMARY KEY (column_name1,
column_name2);
-- (OR)
ALTER TABLE table_name ADD PRIMARY KEY (column1, column2);

-- Adding Primary Key with Auto-Increment
ALTER TABLE table_name MODIFY column_name INT AUTO_INCREMENT PRIMARY KEY;
```

Delete primary Key

```
ALTER TABLE table_name DROP PRIMARY KEY;

-- For multiple column
ALTER TABLE Employee DROP CONSTRAINT PK_Employee;
-- (OR)
ALTER TABLE table_name DROP PRIMARY KEY;
```

Ques. What Is Unique Key?

- A Unique Key is a constraint that ensures all values in a column or a combination of columns are **unique across all records** in a table.
- The Unique and Primary Key constraints both provide a guarantee for a column or set of columns.
- A Primary Key consist automatically has a unique constraint define on it.
 - Defining unique key

```
-- Single Column Unique Key
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    email VARCHAR(100) UNIQUE
);

-- Multiple Column Unique Key
CREATE TABLE Employees (
    id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    UNIQUE (first_name, last_name)
);
```

ALTER unique key

```
-- Single Column
ALTER table Employee ADD UNIQUE(column name);
-- (OR)
ALTER TABLE table_name ADD CONSTRAINT constraint_name UNIQUE (column_name);

-- Multiple Columns
ALTER TABLE table_name ADD CONSTRAINT constraint_name UNIQUE (column1, column2);
```

Drop unique key

```
ALTER TABLE Employee DROP CONSTRAINT Employee_ID;
-- (OR)
ALTER TABLE table_name DROP INDEX constraint_name;
```

Ques. What Is Foreign Key?

- A foreign key is a key used to link two tables together. This is something called a reference key.
- A column or set of columns in a table that references the PRIMARY KEY of another table.
- Foreign key is a column or a combination of columns whose values match a primary key in a different table.
- The relationship between two tables matches the primary key in one of the tables with a foreign key in the second table.

```
-- create Customers table
CREATE TABLE Customers (
  id INTEGER PRIMARY KEY,
  name VARCHAR(100),
  age INTEGER
);

-- create Products table
CREATE TABLE Products (
  customer_id INTEGER ,
  name VARCHAR(100),
  FOREIGN KEY (customer_id)
  REFERENCES Customers(id)
);

-- Here, the customer_id column in the Products table references the id column in
the Customers table.
```

- **One-to-One:-** Each record in one table connects to single record in another

```
CREATE TABLE Employee (
  emp_id INT PRIMARY KEY,
  passport_number INT,
  FOREIGN KEY (passport_number)
  REFERENCES Passport(passport_number)
);
```

- **One-to-Many:-** One record in parent table can relate to multiple records in child table

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  FOREIGN KEY (customer_id)
  REFERENCES Customers(customer_id)
);
```

- **Many-to-Many:-** Multiple records in both tables can relate to each other

```
CREATE TABLE StudentCourses (  
    student_id INT,  
    course_id INT,  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

Alter Foreign Key to Existing Table?

```
-- if primary key doesn't exists in the created table  
ALTER TABLE Employee ADD FOREIGN KEY (department_id) REFERENCES  
Department(department_id);  
(OR)  
ALTER TABLE `bookings` ADD CONSTRAINT `advance_bookings_user_id_foreign` FOREIGN  
KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE ON UPDATE CASCADE  
  
-- For multiple column  
ALTER TABLE Employee ADD CONSTRAINT FK_dept_id FOREIGN KEY (department_id)  
REFERENCES Department(department_id);
```

DROP a Foreign Key from the table

```
-- For single column/multiple column  
ALTER TABLE Employee DROP FOREIGN KEY FK_dept_id;
```

Ques. What is Composite Key?

- Composite key is **combination of two or more columns** that can **uniquely identify each row in the table**.
- composite key is also a primary key, but the difference is that it is made by the combination of more than one column to identify the particular row in the table.
- A composite key cannot be null.

```
CREATE TABLE student
(rollNumber INT,
name VARCHAR(30),
class VARCHAR(30),
section VARCHAR(1),
mobile VARCHAR(10),
PRIMARY KEY (rollNumber, mobile));
```

Types of Composite Keys

1. Composite Primary Key

```
CREATE TABLE StudentCourses (
    student_id INT,
    course_id INT,
    semester VARCHAR(10),
    PRIMARY KEY (student_id, course_id, semester)
);
```

2. Composite Unique Key

```
CREATE TABLE Employees (
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department VARCHAR(50),
    UNIQUE (first_name, last_name, department)
);
```

Can any one field in a composite key be NULL?

- **Composite Primary Key:** No, because all columns that are part of a primary key are automatically NOT NULL.
- **Composite Unique Key:** Yes, in MySQL NULL values can be allowed depending on the constraint and database behaviour.

Ques. Difference between Primary Key & Unique Key?

Primary Key	Unique Key
A table can have only one primary key	A table can have more than one unique key
It does not allow null values	Allows null values
Primary key Can be made foreign key into another table	In SQL server, unique key Can be made foreign key into another table
By default it adds a clustered index	By default it adds a unique non-clustered index
Primary key support auto increment value.	Unique constraint does not support auto increment value.

Ques. Difference between Primary Key & Foreign Key?

Primary Key	Foreign Key
A table can have only one primary key	A table can have more than one foreign key
Primary key uniquely identified a record in the table	Foreign key is a field in the table that is primary key in another table
It does not allow null values	Allows null values
Duplicate not allowed	Duplicate allowed
Primary key support auto increment value	Foreign key do not automatically create an index.

Ques. What Is Joins?

- A JOIN is a method used to **combine rows from two or more tables** based on a related column between them.
- MySQL JOINS are used with SELECT statement.

Many types of MySQL joins:

1. Self Join
2. Inner Join
3. Left JOIN
4. Right JOIN
5. Full Join
6. Outer Join
7. Cross Join

Self Join

- A self join **connects a table to itself**. Used when you want to compare rows within the same table.

Customers table:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

-- Example

```
SELECT c1.NAME AS Customer1, c2.NAME AS Customer2, c1.SALARY FROM Customers c1
JOIN Customers c2 ON c1.SALARY = c2.SALARY AND c1.ID < c2.ID;
```

-- Output:

Customer1	Customer2	SALARY
Ramesh	kaushik	2000.00

INNER JOIN

- The MySQL Inner Join is used to returns only those results from the tables that **match** the specified condition and hides other rows and columns.
- Inner join: Inner join return rows when there is at least one match of rows between the tables.

Customers table:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Order table:

OID	DATE	CUSTOMER_ID	AMOUNT
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

```
SELECT customers.name, customers.age, customers.salary, order.date
FROM customers INNER JOIN order
ON customers.id = order.CUSTOMER_ID;
```

NAME	AGE	SALARY	DATE
Khilan	25	1500.00	2009-11-20 00:00:00
Chaitali	25	6500.00	2008-05-20 00:00:00

-- Example 2

Order table:

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

-- Output:-

```
SQL> SELECT ID, NAME, AMOUNT, DATE FROM CUSTOMERS
INNER JOIN ORDERS
ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

```
+-----+-----+-----+-----+
| ID | NAME      | AGE | AMOUNT |
+-----+-----+-----+-----+
| 3 | kaushik   | 23  | 3000   |
| 3 | kaushik   | 23  | 1500   |
| 2 | Khilan    | 25  | 1560   |
| 4 | Chaitali  | 25  | 2060   |
+-----+-----+-----+-----+
```

Left JOIN/LEFT OUTER JOIN

- The LEFT JOIN keyword returns all records from the left table (the table listed first), and the matching records (if any) from the right table (table2).

CUSTOMERS Table

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Orders Table

OID	DATE	CUSTOMER_ID	AMOUNT
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

```
SQL> SELECT ID, NAME, AMOUNT, DATE FROM CUSTOMERS LEFT JOIN ORDERS
      ON CUSTOMERS.ID = ORDERS.CUSTOMER_ID;
```

Result:-

ID	NAME	AMOUNT	DATE
1	Ramesh	NULL	NULL
2	Khilan	1560	2009-11-20 00:00:00
3	kaushik	3000	2009-10-08 00:00:00
3	kaushik	1500	2009-10-08 00:00:00
4	Chaitali	2060	2008-05-20 00:00:00
5	Hardik	NULL	NULL
6	Komal	NULL	NULL
7	Muffy	NULL	NULL

Right JOIN

- The RIGHT JOIN keyword returns **all records** from the **right table** (table2), and the **matching records** (if any) from the **left table** (table1).

```
-- Customers Table
```

```
+-----+-----+
| ID | NAME      |
+-----+-----+
|  1 | Ramesh    |
|  2 | Khilan    |
|  3 | Kaushik   |
```

```
-- Orders Table
```

```
+-----+-----+
| OrderID | CustomerID |
+-----+-----+
|    101  |          2  |
|    102  |          3  |
|    103  |          4  |
+-----+-----+
```

```
-- RIGHT JOIN Result
```

```
SELECT Customers.NAME, Orders.OrderID FROM Customers RIGHT JOIN Orders
ON Customers.ID = Orders.CustomerID;
```

```
+-----+-----+
| NAME      | OrderID  |
+-----+-----+
| Khilan    |    101   |
| Kaushik   |    102   |
| NULL      |    103   |
+-----+-----+
```

Outer join

- Returns all rows from both tables, Matching and non-matching rows.
- NULL values where no match exists

```
SELECT column_list
FROM table1
FULL OUTER JOIN table2 ON table1.column = table2.column;
```

-- Example query
employees:

employee_id	employee_name	department_id
1	John Smith	101
2	Mary Johnson	102
3	Sam Brown	103

departments:

department_id	department_name
101	HR
102	Finance
104	Marketing

```
SELECT employees.employee_id, employees.employee_name, departments.department_name
FROM employees
FULL OUTER JOIN departments ON employees.department_id =
departments.department_id;
```

employee_id	employee_name	department_name
1	John Smith	HR
2	Mary Johnson	Finance
3	Sam Brown	NULL
NULL	NULL	Marketing

CROSS Join

- The CROSS JOIN keyword returns all records from both tables (table1 and table2).
- If you have a Products table with 2 products and a Colors table with 2 colors, a CROSS JOIN would return a result set with 6 combinations (3 * 2):

-- Products table

ProductID	ProductName
1	T-Shirt
2	Jeans
2	Cap

-- Colors table

ColorID	Color
1	Red
2	Blue

-- Output:-

```
SELECT Products.ProductName, Colors.Color
FROM Products
CROSS JOIN Colors;
```

ProductName	Color
T-Shirt	Red
T-Shirt	Blue
Jeans	Red
Jeans	Blue
Cap	Red
Cap	Blue

Full Join/FULL OUTER JOIN

- A FULL JOIN (also called a FULL OUTER JOIN) returns all rows when there is a match in either left (table1) or right (table2) table. It returns all records from both tables, and the result set will have NULL values for columns where there is no match.

Key Points:

- Rows from both tables are included even if there is no match.
- If there is no match, the columns from the table that doesn't have a match will contain NULL.
- This join is useful when you want to retrieve all records, whether or not there's a match in both tables.

-- Employees Table:

EmployeeID	EmployeeName	DepartmentID
1	John	10
2	Jane	20
3	Mike	30
4	Sara	NULL

-- Departments Table:

DepartmentID	DepartmentName
10	HR
20	IT
30	Sales
40	Marketing

-- Output:-

```
SELECT Employees.EmployeeID, Employees.EmployeeName, Employees.DepartmentID,
Departments.DepartmentName
```

```
FROM Employees
```

```
FULL JOIN Departments
```

```
ON Employees.DepartmentID = Departments.DepartmentID;
```

EmployeeID	EmployeeName	DepartmentID	DepartmentName
1	John	10	HR
2	Jane	20	IT
3	Mike	30	Sales
4	Sara	NULL	NULL
NULL	NULL	40	Marketing